



Calculateur IMC

Rapport Technique — Application Cordova

HTML5 · CSS3 · JavaScript Vanilla — Groupe 3 — ESP/UCAD 2026



UNIVERSITÉ CHEIKH ANTA DIOP DE DAKAR
École Supérieure Polytechnique — ESP

RAPPORT TECHNIQUE

Calculateur IMC

Application Mobile — Apache Cordova

Documentation complète — HTML · CSS · JavaScript

Explication étape par étape de chaque section, fonction et composant visuel

3 sections HTML
structurées

CSS violet thème sombre

8 fonctions JS
documentées

Cordova mobile-first

Auteur

Salif Biaye

Fondateur / CEO — DIC3 Informatique & Télécommunications

Application :	Calculateur IMC
Technologie :	Apache Cordova · HTML5 · CSS3 · JavaScript Vanilla
Module :	Développement Mobile — DIC3 Informatique & Télécommunications
Université :	UCAD — École Supérieure Polytechnique (ESP)
Auteur :	Salif Biaye — Édition Mai 2026

Sommaire

N°	Section	Contenu
1	Vue d'ensemble	Objectif, tech stack, fonctionnement global
2	Structure HTML	Sections , ,
3	Feuille de style CSS	Palette violette, thème sombre, SVG ring
4	Logique JavaScript	Module pattern, 8 fonctions documentées
5	Captures d'écran	Formulaire, résultat IMC, états d'erreur
6	Synthèse	Points clés, bilan technique

SECTION 1

Vue d'ensemble

Objectif de l'application, pile technologique et fonctionnement global

1.1 Qu'est-ce que le Calculateur IMC ?

Le **Calculateur IMC** (Indice de Masse Corporelle) est une application mobile développée avec **Apache Cordova** permettant à un utilisateur d'évaluer sa composition corporelle en saisissant son poids (masse en kg) et sa taille (en mètres). L'IMC est calculé selon la formule : **IMC = masse ÷ taille²**.

Le résultat est affiché accompagné d'une **catégorie de santé** (insuffisance pondérale, poids normal, surpoids, obésité) et d'un **conseil personnalisé**. Un anneau SVG animé visualise graphiquement la valeur obtenue.

1.2 Pile technologique

Technologie	Rôle	Version / Source
Apache Cordova	Framework mobile hybride — wrapper natif Android/iOS	Dernière stable
HTML5	Structure de l'interface (sémantique, formulaire, SVG)	Standard W3C
CSS3	Mise en page, thème violet sombre, animations, mode clair	Standard W3C
JavaScript Vanilla	Logique métier, calcul IMC, rendu dynamique	ES6+
Font Awesome 6.5	Icônes vectorielles (balance, règle, flèche)	CDN Cloudflare

1.3 Flux utilisateur

- **Étape 1 — Saisie** : l'utilisateur entre sa masse (kg) et sa taille (m) dans le formulaire.
- **Étape 2 — Validation** : JavaScript vérifie les bornes (1–500 kg, 0.5–3 m).
- **Étape 3 — Calcul** : $IMC = \text{masse} / (\text{taille} \times \text{taille})$, arrondi à 1 décimale.
- **Étape 4 — Affichage** : la carte résultat apparaît avec la valeur, la catégorie colorée et le conseil.
- **Étape 5 — Animation** : l'anneau SVG se remplit proportionnellement à l'IMC (max 40).

Synthèse
☐ Framework : Apache Cordova — code web converti en application Android/iOS
☐ Calcul : $IMC = \text{masse} / \text{taille}^2$ — formule OMS standard
☐ Retour visuel : anneau SVG animé + catégorie colorée + conseil textuel

☐ Thème : violet foncé (#2E1065 → #8B5CF6) avec support mode clair

SECTION 2

Structure HTML

Analyse détaillée de chaque balise et section du fichier `index.html`

2.1 En-tête du document ()

Le `<head>` configure les métadonnées essentielles pour une application mobile Cordova :

□ `index.html` — `head`

```
<meta name="viewport" content="width=device-width, initial-scale=1.0, user-scalable=no">

<!-- Désactive le zoom utilisateur pour une UX d'app native -->

<meta http-equiv="Content-Security-Policy" content="default-src 'self' ...">

<!-- Politique de sécurité Cordova : whitelist des ressources autorisées -->

<link rel="stylesheet" href="https://cdnjs.cloudflare.com/...font-awesome...css">

<link rel="stylesheet" href="css/index.css">
```

- **viewport user-scalable=no** : empêche le zoom, simule le comportement d'une app native.
- **Content-Security-Policy** : directive de sécurité Cordova, liste les sources autorisées.
- **Font Awesome** : chargé depuis CDN pour les icônes (balance, règle, flèche).

2.2 Conteneur principal (#app)

Tout le contenu visible est encapsulé dans `<div id="app">`, racine CSS qui applique le thème global (fond sombre, police, max-width).

□ `index.html` — `structure générale`

```
<div id="app">

  <header> ... </header>      <!-- Bande titre dégradé violet -->

  <div class="content">      <!-- Zone scrollable -->

    <form id="imc-form"> ... </form>  <!-- Formulaire saisie -->

    <div id="result" hidden> ... </div> <!-- Carte résultat -->

  </div>

</div>
```

2.3 En-tête visuelle ()

L'en-tête contient le titre de l'application et un sous-titre descriptif, stylisés avec le dégradé violet.

□ `index.html` — `header`

```
<header>

  <div class="header-inner">

    <h1>Calculateur IMC</h1>

    <p class="subtitle">Évaluez votre composition corporelle en quelques secondes.</p>

  </div>

</header>
```

2.4 Formulaire de saisie (#imc-form)

Le formulaire est déclaré avec l'attribut `novalidate` — la validation est entièrement gérée en JavaScript pour un contrôle total sur les messages d'erreur. La grille 3 colonnes (masse / séparateur / taille) est structurée par `.field-group`.

□ index.html — form

```
<form id="imc-form" novalidate>

  <div class="field-group">      <!-- Grille 3 colonnes CSS -->

    <div class="field">

      <label for="masse"><i class="fa-solid fa-weight-scale"></i> Masse <span class="unit">kg</span></label>

      <input type="number" id="masse" placeholder="70" min="1" max="500" step="0.1" inputmode="decimal" required>

      <span class="field-hint">Entre 1 et 500 kg</span>

    </div>

    <div class="field-divider"></div>  <!-- Séparateur visuel -->

    <div class="field"> ... taille ... </div>

  </div>

  <button type="submit" class="btn-submit">

    <span>Calculer l'IMC</span> <i class="fa-solid fa-arrow-right"></i>

  </button>

</form>
```

- `type="number"` : clavier numérique sur mobile.
- `inputmode="decimal"` : clavier décimal (virgule disponible) sur iOS/Android.
- `min/max` : bornes HTML natives (doublées en JS pour fiabilité).
- `step="0.1"` / `step="0.01"` : précision adaptée à chaque champ.

2.5 Carte résultat (#result)

La carte est initialement cachée (`hidden`). Elle est affichée par JS après calcul. Elle contient un **anneau SVG**, la valeur numérique de l'IMC, la catégorie et le conseil.

index.html — result card

```
<div id="result" class="result" aria-live="polite" hidden>
  <div class="result-ring">
    <svg class="ring-svg" viewBox="0 0 120 120">
      <circle class="ring-bg" cx="60" cy="60" r="50"/> <!-- Fond gris -->
      <circle class="ring-fill" cx="60" cy="60" r="50"/> <!-- Arc animé -->
    </svg>
    <div class="result-value-wrap">
      <span id="result-value">--</span> <!-- Valeur IMC -->
      <span class="result-label-imc">IMC</span>
    </div>
  </div>
  <p id="result-category">--</p> <!-- Catégorie colorée -->
  <p id="result-advice"></p> <!-- Conseil personnalisé -->
</div>
```

L'attribut `aria-live="polite"` annonce le résultat aux lecteurs d'écran dès qu'il apparaît — bonne pratique d'accessibilité.

Technique SVG — anneau animé : les deux cercles ont le même centre (`cx=60`, `cy=60`) et le même rayon (`r=50`). Le cercle `.ring-fill` utilise `stroke-dasharray` et `stroke-dashoffset` pour créer l'arc proportionnel à l'IMC.

Synthèse

- ❑ `novalidate` : validation 100% JS pour maîtriser les messages d'erreur
- ❑ `inputmode=decimal` : clavier adapté sur iOS/Android
- ❑ SVG ring : `stroke-dashoffset` animé par CSS transition 1s ease
- ❑ `aria-live=polite` : accessibilité — annonce vocale du résultat

SECTION 3

Feuille de style CSS

Palette violette, thème sombre, layout et animations SVG

3.1 Palette de couleurs

Variable CSS	Hex	Usage
fond global	#0d0718	Fond arrière très sombre (presque noir violet)
header gradient	#2E1065 → #A78BFA	Dégradé en-tête de haut en bas
accent principal	#6D28D9	Bouton, bordures actives, ring stroke
accent clair	#8B5CF6	Hover, focus, éléments secondaires
texte	#EDE9FE	Texte principal sur fond sombre (lavande))
champ input	#130A24	Fond des inputs (plus sombre que le fond global)
erreur	#FF4D6A	Bordure rouge-rose si validation échoue
IMC normal	#00E87A	Catégorie Poids normal — vert vif
IMC surpoids	#FFB443	Catégorie Surpoids — orange
IMC obésité	#FF4D6A	Catégorie Obésité — rouge-rose

3.2 Layout et structure

L'application utilise un layout **Flexbox** centré, avec une largeur maximale de **480px** pour conserver l'apparence mobile sur grand écran. La zone de contenu est scrollable, le header reste fixe en haut.

index.css – layout principal

```
#app {
  display: flex; flex-direction: column;
  min-height: 100vh;
  background: #0d0718; color: #ede9fe;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
}

header { background: linear-gradient(160deg, #2e1065, #6d28d9, #a78bfa); }

.field-group {
  display: grid;
  grid-template-columns: 1fr auto 1fr; /* Masse | / | Taille */
}
```



```
gap: 12px; align-items: center;
}
```

3.3 Animation de l'anneau SVG

L'effet d'anneau progressif est obtenu en manipulant les propriétés SVG `stroke-dasharray` et `stroke-dashoffset` via CSS et JavaScript.

□ `index.css — ring SVG`

```
.ring-fill {
  fill: none;

  stroke: #8b5cf6;           /* Couleur par défaut, changée par JS */
  stroke-width: 12;
  stroke-linecap: round;
  stroke-dasharray: 314;     /* 2π × r=50 ≈ 314 — périmètre complet */
  stroke-dashoffset: 314;    /* Anneau invisible au départ */
  transform: rotate(-90deg); /* Commence à midi (haut) */
  transform-origin: center;
  transition: stroke-dashoffset 1s ease, stroke 0.4s ease;
}

/* JS : fill.style.strokeDashoffset = 314 * (1 - imc/40) */
```

3.4 Mode clair (media query)

L'application respecte la préférence système de l'utilisateur via `@media` (`prefers-color-scheme: light`). Les couleurs s'adaptent automatiquement sans JavaScript.

□ `index.css — light mode`

```
@media (prefers-color-scheme: light) {
  #app { background: #f5f3ff; color: #1e0a44; }
  header { background: linear-gradient(160deg, #4c1d95, #7c3aed, #a78bfa); }
  .field input { background: #ffffff; color: #1e0a44; border-color: #c4b5fd; }
  .result { background: #ede9fe; }
}
```

Synthèse

- Palette : violet foncé #0d0718 → lavande #a78bfa — 5 tons
- Grid 3 colonnes : mise en page originale masse/séparateur/taille

☐ SVG ring : stroke-dashoffset animé (1s ease) via calcul JS

☐ Mode clair automatique via prefers-color-scheme — aucun JS nécessaire

SECTION 4

Logique JavaScript

Module Pattern IIFE — 8 fonctions documentées étape par étape

4.1 Module Pattern et constantes

Le code est encapsulé dans un **IIFE (Immediately Invoked Function Expression)** qui retourne uniquement la méthode `init`. Ce pattern évite de polluer le scope global et structure proprement le code.

□ `index.js` — structure globale

```
var App = (function () {
  // Constantes privées

  var RING_MAX = 40; // IMC max pour le remplissage complet de l'anneau
  var RING_LEN = 314; // 2 × π × r=50 — périmètre SVG en px

  // Tableau de catégories IMC
  var CATEGORIES = [ ... ];

  // Fonctions privées
  function getCategory(imc) { ... }
  function validate(masse, taille) { ... }
  // ...

  // API publique
  return { init: init };
})();

App.init(); // Démarrage de l'application
```

4.2 Tableau CATEGORIES

Le tableau `CATEGORIES` contient 5 objets décrivant les plages IMC, chacun avec un seuil max, un label à afficher, un advice personnalisé et une color hex.

Catégorie	Seuil max	Couleur	Conseil
Insuffisance pondérale	18.5	#FFB443 (orange)	Consultez un professionnel de santé
Poids normal	25	#00E87A (vert)	Continuez vos bonnes habitudes !
Surpoids	30	#FFB443 (orange)	Activité physique recommandée
Obésité modérée	35	#FF7043 (rouge-or)	Suivi médical recommandé
Obésité sévère	∞	#FF4D6A (rouge-rose)	Consultez rapidement

4.3 getCategory(imc)

Retourne le premier objet CATEGORIES dont le max est supérieur à l'IMC. Utilise `Array.find()` — s'arrête au premier match.

□ `index.js` — `getCategory`

```
function getCategory(imc) {
  return CATEGORIES.find(function (c) {
    return imc < c.max; // Premier seuil dépassé = bonne catégorie
  });
}

// Exemple : getCategory(22.5) → { label: "Poids normal", color: "#00e87a", ... }
```

4.4 validate(masse, taille)

Contrôle les bornes des deux champs. Retourne un tableau des IDs en erreur (vide si tout est valide). Double validation : HTML min/max + JS.

□ `index.js` — `validate`

```
function validate(masse, taille) {
  var errors = [];
  if (!masse || masse < 1 || masse > 500) errors.push("masse");
  if (!taille || taille < 0.5 || taille > 3) errors.push("taille");
  return errors; // [] = valide, ["masse"] = erreur masse, etc.
}
```

4.5 showErrors(fields)

Applique la classe `.error` aux champs invalides (bordure rouge, fond rose). Réinitialise d'abord toutes les erreurs avant d'appliquer les nouvelles.

□ `index.js` — `showErrors`

```
function showErrors(fields) {
  // Reset : supprimer toutes les erreurs existantes
  document.getElementById("masse").classList.remove("error");
  document.getElementById("taille").classList.remove("error");

  // Appliquer les nouvelles erreurs
  fields.forEach(function (id) {
    document.getElementById(id).classList.add("error");
  });
}
```

```
});  
  
}
```

4.6 animateRing(imc, color)

Anime l'anneau SVG en calculant le stroke-dashoffset proportionnel à l'IMC. Un délai de 50ms assure que le DOM est prêt avant l'animation.

□ index.js — animateRing

```
function animateRing(imc, color) {  
  var fill = document.querySelector(".ring-fill");  
  // Calcul : offset = RING_LEN * (1 - proportion)  
  // proportion = imc / RING_MAX (plafonné à 1)  
  var offset = RING_LEN * (1 - Math.min(imc / RING_MAX, 1));  
  fill.style.stroke = color; // Couleur selon catégorie  
  setTimeout(function () {  
    fill.style.strokeDashoffset = offset; // Déclenche l'animation CSS 1s  
  }, 50);  
}  
  
// Exemple : IMC=25 → offset=314*(1-25/40)=314*0.375=117.75
```

4.7 showResult(imc, category)

Met à jour les éléments DOM avec la valeur, la catégorie et le conseil, affiche la carte résultat, puis la fait défiler en vue.

□ index.js — showResult

```
function showResult(imc, category) {  
  // Afficher la valeur arrondie à 1 décimale  
  document.getElementById("result-value").textContent = Math.round(imc * 10) / 10;  
  // Catégorie avec sa couleur  
  document.getElementById("result-category").textContent = category.label;  
  document.getElementById("result-category").style.color = category.color;  
  // Conseil personnalisé  
  document.getElementById("result-advice").textContent = category.advice;  
  // Révéler la carte résultat  
  var resultDiv = document.getElementById("result");  
  resultDiv.removeAttribute("hidden");  
  resultDiv.scrollIntoView({ behavior: "smooth", block: "nearest" });  
}
```

4.8 onSubmit(e) — orchestrateur

Fonction principale déclenchée à la soumission du formulaire. Elle coordonne validation → calcul → affichage → animation.

□ index.js — onSubmit

```
function onSubmit(e) {
  e.preventDefault(); // Empêche le rechargement de la page

  var masse = parseFloat(document.getElementById("masse").value);
  var taille = parseFloat(document.getElementById("taille").value);
  var errors = validate(masse, taille);

  showErrors(errors);

  if (errors.length) return; // Stop si erreurs

  var imc = masse / (taille * taille); // Formule IMC
  var category = getCategory(imc);

  showResult(imc, category); // Afficher la carte
  animateRing(imc, category.color); // Animer le SVG
}
```

4.9 bindEvents() et init()

□ index.js — bindEvents / init

```
function bindEvents() {
  // Écouter la soumission du formulaire
  document.getElementById("imc-form").addEventListener("submit", onSubmit);
}

function init() {
  bindEvents(); // Attacher tous les événements
}

return { init: init }; // Exposer uniquement init

// Démarrage immédiat
App.init();
```

Synthèse

□ IIFE Module : scope privé, aucune variable globale exposée

- ❑ CATEGORIES[] : configuration centralisée des 5 pages IMC
- ❑ validate() : retourne un tableau d'IDs en erreur pour un reset propre
- ❑ animateRing() : setTimeout 50ms pour laisser le DOM se mettre à jour
- ❑ onSubmit() : orchestrateur — 4 étapes séquentielles claires

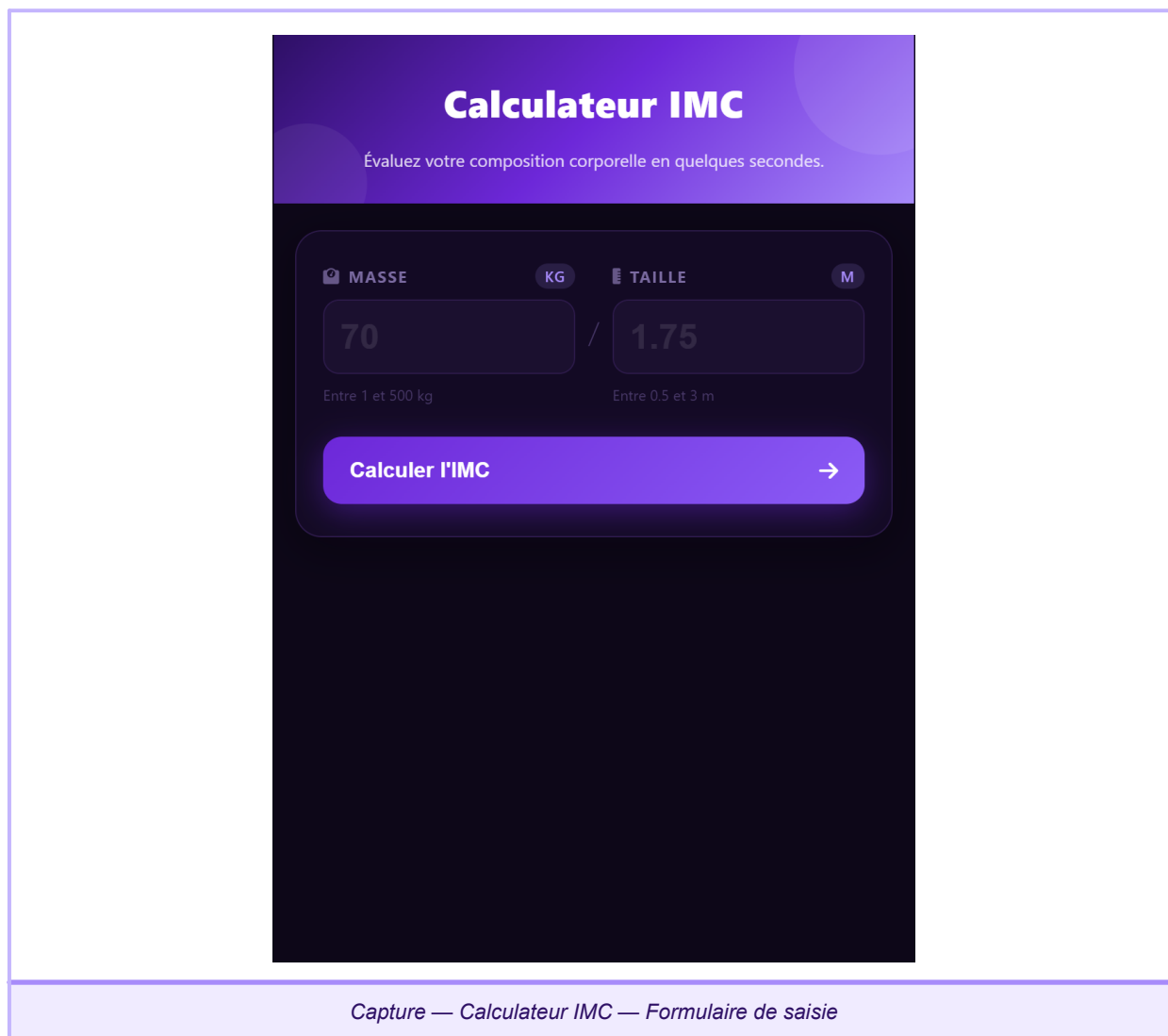
SECTION 5

Captures d'écran

Placer les images dans : C:\Users\DELL\Downloads\medy\screens

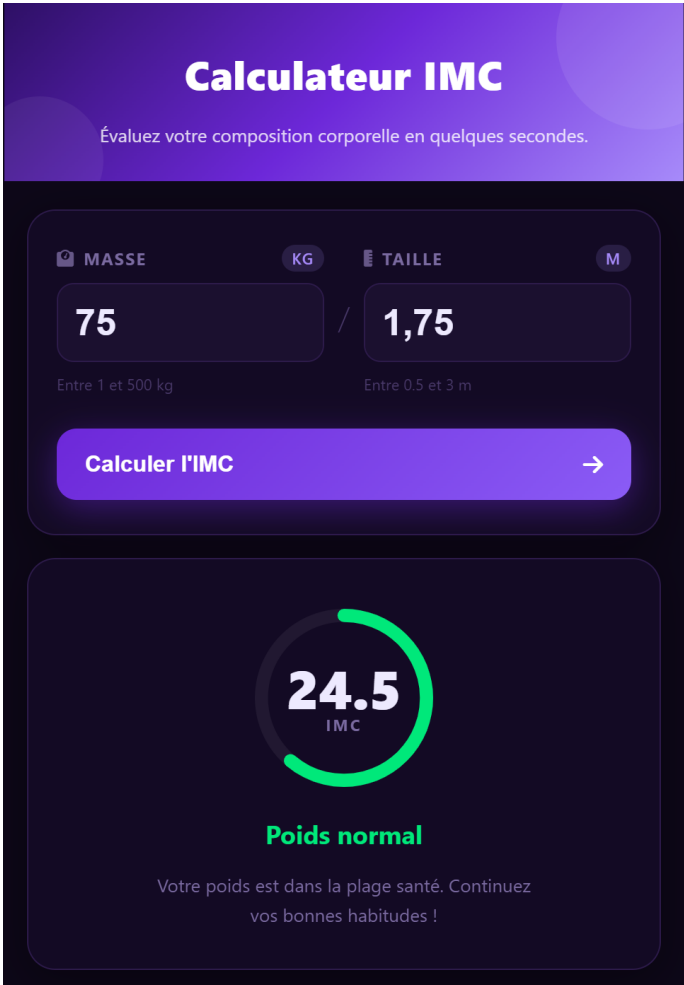
5.1 Formulaire de saisie

Vue initiale : en-tête violet, deux champs de saisie, bouton de calcul.



5.2 Résultat IMC — Poids normal

Carte résultat affichée : anneau SVG vert animé, valeur IMC, catégorie colorée, conseil.



Calculateur IMC

Évaluez votre composition corporelle en quelques secondes.

MASSE **KG** **TAILLE** **M**

75 / **1,75**

Entre 1 et 500 kg Entre 0.5 et 3 m

Calculer l'IMC →

24.5
IMC

Poids normal

Votre poids est dans la plage santé. Continuez vos bonnes habitudes !

Capture — Calculateur IMC — Résultat (poids normal)

5.3 État d'erreur

Validation échouée : bordure rouge sur les champs invalides.

Calculateur IMC

Évaluez votre composition corporelle en quelques secondes.

MASSE

KG

0

Entre 1 et 500 kg

TAILLE

M

0

Entre 0.5 et 3 m

Calculer l'IMC →

Capture — Calculateur IMC — Erreur de validation

SECTION 6

Synthèse finale

Bilan technique, points forts et compétences démontrées

6.1 Bilan technique

Aspect	Détail	Niveau
Architecture JS	IIFE Module Pattern — scope isolé, API minimale	☐ Solide
HTML sémantique	form, label, button, aria-live — accessibilité native	☐ Bon
CSS avancé	Grid 3 col, SVG animation, media query light mode	☐ Avancé
Validation	Double validation HTML + JS avec retour visuel clair	☐ Complet
UX mobile	inputmode, viewport, scroll smooth, animation fluide	☐ Optimisé
Cordova	deviceready stub, CSP configurée, structure www/	☐ Conforme

6.2 Compétences démontrées

- Maîtrise du **JavaScript Vanilla** sans dépendances (pas de jQuery, pas de framework).
- Utilisation avancée du **SVG inline** avec animation via stroke-dashoffset.
- Connaissance des **patterns de conception** (Module Pattern / IIFE).
- Bonne pratique **d'accessibilité** (aria-live, labels associés, inputmode).
- Respect des **contraintes Cordova** (CSP, structure www/, deviceready).

Synthèse

- ☐ 8 fonctions JS documentées : getCategory, validate, showErrors, animateRing, showResult, onSubmit, bindEvents, init
- ☐ 3 sections HTML : header, form (field-group), result (SVG + texte)
- ☐ CSS : thème violet, grid layout, SVG ring, mode clair automatique
- ☐ Zéro dépendance JS : application 100% Vanilla JavaScript

Calculateur IMC

Rapport Technique — Application Cordova

UCAD — École Supérieure Polytechnique (ESP)

Salif Biaye — Fondateur / CEO

Module : Développement Mobile — DIC3 — Édition Mai 2026

Document produit dans le cadre du cursus DIC3 — Usage académique uniquement